

TECHNICAL DEEP-DIVE • 20 MIN

From Prompt to Production

The End-to-End AI Workflow

How a request travels from a single LLM call to an autonomous, multi-agent system — and the engineering at every layer.





THE SHIFT

A chatbot answers. An agent acts.

The last two years moved us from “LLM as a text box” to LLMs that plan, call tools, and complete multi-step work. The skill now isn’t writing one clever prompt — it’s engineering the workflow around the model.



YESTERDAY

Single-call chatbot

- One prompt → one response
- Stateless, no memory
- Can’t act on the world
- Knowledge frozen at training



TODAY

Agentic workflow

- Reasons in a loop, multi-step
- Remembers + retrieves your data
- Calls tools, APIs, other agents
- Observable, guardrailed, evaluated



THE MAP

The workflow, in 8 layers

Every production AI system is built up this stack. Today we climb it bottom-to-top — and demo the agentic core.

1

LLM Foundations

Next-token predictor. Stochastic by design.



2

Prompt Engineering

Vague task -> reliable, structured output.



3

APIs & SDKs

Streaming, async, retries, cost tracking.



4

Tool Use

LLM calls your functions. Agentic begins.



5

RAG & Vector DBs

Answer from YOUR data, not training data.



6

Agent Patterns

ReAct, Plan-Execute, Reflection, Multi-agent.



7

Frameworks

LangGraph, MCP, CrewAI — graduate to these.



8

Production / LLMOps

Observability, guardrails, eval, cost.





The model + how you talk to it

LAYER 1 — WHAT AN LLM ACTUALLY IS

- A next-token predictor — not a database, not a reasoner.
- Output is stochastic: same prompt can vary (temperature).
- No memory between calls — you supply all context every time.
- Confidently wrong = hallucination. Plan for it.

*Engineering takeaway: treat the model as an unreliable function.
Constrain it, verify it, give it the context it needs.*

LAYER 2 — PROMPTING = RELIABILITY

- System prompt = role + rules + output contract.
- Few-shot + chain-of-thought for harder reasoning.
- Force structured output (JSON / Pydantic) — parse, don't hope.
- The goal: a vague task → the same valid shape, every time.

```
contract
response_format = MyModel # typed
temperature = 0           # deterministic
```



Same model, real engineering

The model is the easy part. Shipping it means handling latency, failure, concurrency, and cost — the same problems as any distributed system.



Streaming

Token-by-token output.
Perceived latency drops; UX feels instant.



Async & Parallel

Fan out N calls concurrently. 10x throughput on batch work.



Retries & Backoff

Rate limits + timeouts are normal. Exponential backoff, idempotency.



Cost Tracking

Tokens = money. Meter per request; pick the cheapest model that passes.

Rule of thumb: a demo ignores all four. Production lives or dies on them.



Tool use: the LLM acts

- You hand the model a menu of functions (name + schema + when-to-use).
- It picks the tool and fills the arguments — you execute the code.
- Result goes back in; it decides the next step.
- Tool description quality = agent reliability. Write them like docs.

This single capability — call a function, read the result, decide again — is the seed of every agent.

THE TOOL-USE LOOP

1 • LLM decides

`"I should call get_weather"`



2 • Tool executes

`get_weather('Mumbai') → 32°C`



3 • Observation in

`feed result back to the model`



4 • Answer or loop

`done? respond. else → step 1`



RAG: ground the model in truth

LLMs don't know your docs, your tickets, or last week's release. RAG retrieves the right chunks and puts them in context — cutting hallucination and citing sources.



Quality levers: chunking strategy · embedding model · hybrid search · reranking · *measure with RAGAS (faithfulness, relevance).*



Agent design patterns

Reusable shapes for orchestrating reasoning. Most real systems combine two or three.



ReAct

DEMO

Reason → act → observe, in a loop. The default agent.



Plan & Execute

Make a full plan first, then run each step.



Reflection

Critique own output, then revise. Quality boost.



Multi-Agent

DEMO

Supervisor delegates to specialist workers.



Routing

Classify the request, send to the right path/model.



Human-in-Loop

Pause for approval on risky / costly actions.



THE KEY DISTINCTION

Workflows vs. Agents

Anthropic's framing — and the single most useful design decision you'll make.

WORKFLOWS

LLM steps wired by predefined code paths.

- Prompt chaining — step → step → step
- Routing — classify, then branch
- Parallelization — fan out, aggregate
- Orchestrator-workers · Evaluator-optimizer

Predictable, cheap, debuggable. Start here.

AGENTS

LLM directs its own process & tool use.

- Open-ended — no fixed path
- Loops until done (or budget hit)
- Handles ambiguity & dynamic steps
- Needs guardrails, limits, observability

Powerful, costlier, harder to predict.

Golden rule: use the simplest thing that works. Reach for an agent only when the path can't be hard-coded.



ZOOM IN

ReAct: the loop we'll build live

LAYER 6 OF 8

Reasoning + Acting interleaved. The model writes its thought, names a tool, reads the result, and repeats until it can answer — all driven by one structured prompt.

- **No framework needed** — it's a prompt + a parse + a while-loop.
- Stop sequence on "Observation:" hands control back to your code.
- Production adds: max-iterations, timeout, \$ budget, stuck-detection.

react · live transcript

Thought: I need the weather in Mumbai.

Action: get_weather

Action Input: {"city": "Mumbai"}

Observation: 32°C, sunny

Thought: Now compute $25 * 8$.

Action: calculator

Action Input: "25 * 8"

Observation: 200

Thought: I have everything.

Final Answer: Mumbai is 32°C; $25 \times 8 = 200$.

ReAct agent — from scratch, no framework

terminal

```
$ cd Level6_Agent_Patterns  
$ python 04_react_pattern_practical.py
```

WHAT TO WATCH

- The Thought → Action → Observation trace, step by step
- Two tools chained in one question (weather + math)
- Production agent prints cost, iterations, tools used
- Safety stops: timeout · budget · stuck-detection

FILE

```
Level6_Agent_Patterns/  
04_react_pattern_practical.py
```

MAPS TO LAYERS

1 LLM · 2 Prompt · 4 Tools · 6 ReAct

NO API KEY?

File runs in safe mode and prints [NO_API_KEY] — walk the printed trace instead.



Frameworks: when to reach for them

Build the loop by hand first — you'll understand what frameworks hide. Then adopt one for state, durability, and standard integrations.

LangGraph

MUST-KNOW

Stateful agent graphs — nodes, edges, checkpoints, human-in-loop.

Use for: *Complex, durable, multi-step workflows.*

MCP

MUST-KNOW

Model Context Protocol — a USB-C standard for tools & data sources.

Use for: *Reusable tools across apps & models.*

CrewAI

Role-based crews of agents with goals and delegation.

Use for: *Fast multi-agent prototypes.*

LangChain • LlamaIndex • Pydantic-AI

Building blocks: chains, RAG plumbing, typed agents.

Use for: *Glue, retrieval, type-safe outputs.*



Production: demos vs. systems

What separates a notebook from a product the team can trust. None of it is the model — all of it is operations.



Observability

Trace every step, token & cost. LangSmith / Langfuse.



Guardrails & Safety

Block prompt injection, PII leaks, unsafe output.



Evaluation

Golden sets + LLM-as-judge. Catch regressions in CI.



Cost Optimization

Cache, route to cheaper models, trim context.



Human-in-Loop

Approval gates for risky or irreversible actions.



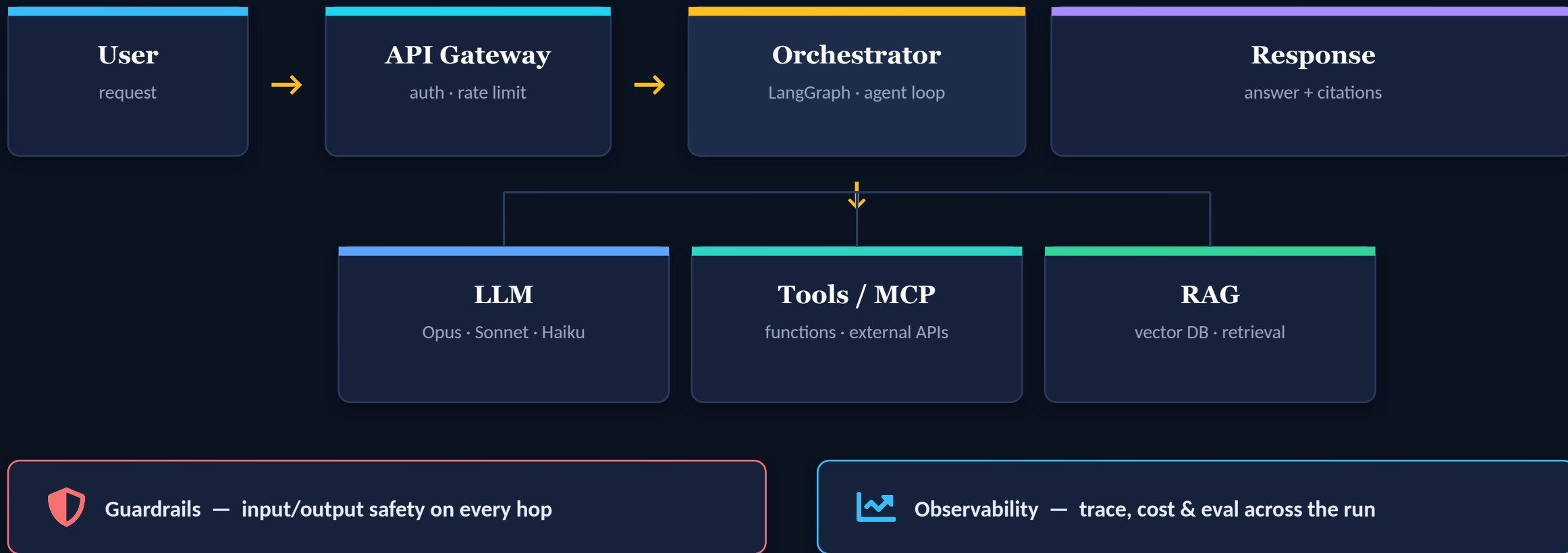
Reliability

Fallbacks, retries, graceful degradation, limits.



PUTTING IT TOGETHER

One end-to-end reference architecture



Multi-agent code review — the whole stack at once

GITHUB PR → PARALLEL SPECIALISTS → SYNTHESIZE → SHIP

Security Agent

Opus

\$0.08 / PR

Performance Agent

Sonnet

\$0.02 / PR

Style Agent

Haiku

\$0.001 / PR

↓ *synthesize* → *critical? human-in-loop* → *post inline comments + Slack alert*

Total ≈ **\$0.10 / PR** — model tiering = 80× cheaper than all-Opus

PATTERNS ON DISPLAY

- Multi-agent supervisor (parallel fan-out)
- Routing by severity
- Human-in-the-loop interrupt
- Cost-aware model tiering
- MCP tool (Slack alert)

`Projects/project3_multiagent_code_review_starter/main.py`

Runs in placeholder mode — prints the architecture even with no key.



WRAP-UP

When to use what — and what to remember

THE COMPLEXITY LADDER — climb only as far as you must

- 1 Single LLM call
- 2 + Structured prompt / output
- 3 + Tools (function calling)
- 4 + RAG (your data)
- 5 + Agent loop (ReAct)
- 6 + Multi-agent + frameworks

Each rung adds power and cost. Most problems stop at rung 3–4.

TAKE THIS HOME

- The model is one layer. The workflow around it is the real work.
- Start with a workflow; reach for an agent only when needed.
- Tools + RAG turn a chatbot into a system that acts on truth.
- Build the loop by hand once — then graduate to LangGraph / MCP.
- Demos skip observability, guardrails, eval, cost. Products can't.

Thank you · Questions?